

Anomaly Detection on Attributed Networks via Multiview and Multiscale Contrastive Learning

Shuxin Qin , Yongcan Luo , *Student Member, IEEE*, Jing Zhu , Gaofeng Tao ,
Jingya Zheng , and Zhongjun Ma 

Abstract—Detecting abnormal nodes from attributed networks plays an important role in various applications, including cybersecurity, finance, and social networks. Most existing methods focus on learning different scales of graphs or using augmented data to improve the quality of feature representation. However, the performance is limited due to two critical problems. First, the high sensitivity of attributed networks makes it uncontrollable and uncertain to use conventional methods for data augmentation, leading to limited improvement in representation and generalization capabilities. Second, under the unsupervised paradigm, anomalous nodes mixed in the training data may interfere with the learning of normal patterns and weaken the discrimination ability. In this work, we propose a novel multiview and multiscale contrastive learning framework to address these two issues. Specifically, a network augmentation method based on parameter perturbation is introduced to generate augmented views for both node–node and node–subgraph level contrast branches. Then, cross-view graph contrastive learning is employed to improve the representation without the need for augmented data. We also provide a cycle training strategy where normal samples detected in the former step are collected for an additional training step. In this way, the ability to learn normal patterns is enhanced. Extensive experiments on six benchmark datasets demonstrate that our method outperforms the existing state-of-the-art baselines.

Index Terms—Anomaly detection, attributed networks, contrastive learning, parameter perturbation.

I. INTRODUCTION

RECENT years have experienced rapid advancements in real-world scenarios leveraging attributed networks, e.g., social networks [1], IoT networks [2], infrastructure networks [3], and financial transaction networks [4]. Anomaly detection from attributed networks holds great importance as

it plays a key role in minimizing risks and preventing adverse events. As we know, an attributed network can be defined as a graph in which nodes and/or edges have attributes. Therefore, anomaly detection on attributed networks is also known as graph anomaly detection (GAD) in most security-related scenarios, such as fraud detection [5], cybersecurity [6], and social spam detection [7]. Generally, GAD aims to identify anomalous nodes whose behaviors significantly differ from those of the majority in attributed networks. Given the intricate topological structure and characteristics of the nodes, the anomalies can be classified into different categories: structural anomalies and contextual anomalies [8]. For example, in social networks, unexpected social activities indicated by node relationships represent structural anomalies, whereas malicious users whose attributes markedly differ from those of their neighbors are contextual anomalies.

GAD is a challenging task due to the complex graph structure, the lack of positive samples, and the high cost of annotations. In the past few years, a considerable number of works [9], [10], [11], [12], [13], [14], [15], [16] have been presented to solve the problem in an unsupervised manner. Earlier research [17], [18], [19] mainly resorts to shallow learning methods to learn graph structures and measure the anomalies. For example, in [18], a learning framework is proposed to characterize attribute reconstruction residuals and correlational patterns with network structure information. Residual errors generated by a regression function are used to detect anomalies. Similarly, attribute and network structure information are used together for simultaneous attribute selection and anomaly detection in [19]. Despite the simplicity of these shallow methods, they cannot capture the complex correlational patterns in the graph data.

Thanks to the rapid development of deep neural networks, recent deep GAD methods have made impressive progress [20], [21]. Graph neural networks (GNNs) and deep graph representation have become indispensable to deal with the high-dimensional and complex correlated graph data [22]. Generally, GNNs learn the representations of a node by aggregating information from its neighboring nodes in the graph. Therefore, both the attributes of the individual nodes and the structure of the entire graph can be easily captured by GNNs. CoLA [8] employs a GNN-based contrastive learning framework to model the relationship between each node and its partial neighboring substructure (node–subgraph contrast). To improve the representation capability and capture the anomalous patterns

Received 24 February 2024; revised 26 October 2024 and 28 November 2024; accepted 5 December 2024. Date of publication 17 December 2024; date of current version 2 June 2025. This work was supported in part by the internal research program of Purple Mountain Laboratories. (Corresponding authors: Shuxin Qin; Gaofeng Tao.)

Shuxin Qin and Jing Zhu are with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: qinshuxin@pmlabs.com.cn).

Yongcan Luo is with the Department of Computer Science, City University of Hong Kong, Hong Kong 999077, China.

Gaofeng Tao is with Purple Mountain Laboratories, Nanjing 211111, China, and also with Shandong Future Network Research Institute, Jinan 250003, China (e-mail: taogaofeng@pmlabs.com.cn).

Jingya Zheng and Zhongjun Ma are with Shandong Future Network Research Institute, Jinan 250003, China.

Digital Object Identifier 10.1109/TCSS.2024.3514148

from different perspectives, ANEMONE [11] adds a node–node contrast branch to detect anomalies in multiple graph scales. GRADATE [23] further improves the representation of local features and structure for subgraphs through contrastive learning from different perspectives. The view is generated by data augmentation, which is an efficient strategy to improve generalization and self-adaption ability. AugAN [24] enhances the model generalization ability of GAD through a well-designed data augmentation method, which improves the quantity of labeled anomalies and enhances the diversity of normal backgrounds. In summary, most existing methods focus on learning different scales of graphs or using augmented data to improve the quality of node embeddings and structure representations.

Despite the remarkable progress in GAD, its performance is still limited due to two critical problems. First, due to the high sensitivity of attributed networks, it is uncontrollable and uncertain to directly apply conventional data augmentation methods, resulting in limited improvement in representation and generalization capabilities. In practice, we usually have to try different data augmentation strategies, which is inefficient and unreliable. Even worse, data augmentation is not always effective. Second, under the unsupervised paradigm, anomalous nodes mixed in the training data may interfere with the learning of normal patterns and thus weaken the discrimination capability to some extent. Considering that anomalies are the nodes deviating from the normal patterns, the performance is limited by the mixed training data.

To solve the above challenges, in this article, we propose a multiview and multiscale contrastive learning framework for graph anomaly detection (MMGAD). It is a multitask learning framework in which the multiview graph contrastive learning (GCL) task is designed as an auxiliary task to the multiscale learning task. Specifically, the multiscale task is used to capture anomalous patterns at the node level and subgraph level by performing node–node and node–subgraph contrasts in parallel. Each contrast branch employs a GNN-based network to learn the latent patterns. The multiview task is designed to improve the representation of the two branches by cross-view GCL. In contrast to existing methods, MMGAD generates augmented views for both node–node and node–subgraph contrasts using a novel network augmentation method. The method is based on parameter perturbation, which avoids modifying the sensitive training data. With the augmented views, GCL can enforce the agreement between positive pairs compared to negative pairs at the cross-view level, resulting in a better representation. We also provide a novel cycle training strategy to enhance the learning of normal patterns and thus improve the discrimination capability. Normal samples recognized with high confidence in the former step are collected as training data for an additional training step. After a certain number of steps, reliable normal samples can be picked out without the help of ground truth labels, and the model can be further refined step by step with the relatively pure training samples.

In summary, the contributions of our work are as follows.

- 1) We develop a novel multiview and multiscale contrastive learning framework, MMGAD, for anomaly detection on attributed networks. It is an unsupervised multitask

learning framework that learns latent patterns at different views and scales.

- 2) We utilize a network augmentation technique to create correlated views for cross-view contrast. In contrast to existing approaches that directly augment graph data, our method relies on parameter perturbation. The cross-view GCL enhances both node–node and node–subgraph branches.
- 3) We propose a cycle training strategy in which normal samples detected in the former step are collected for an additional training step. It is an online sampling and training strategy that can refine the model in a circular manner.
- 4) The proposed method achieves state-of-the-art performance on six public datasets. Ablation study and detailed analysis are provided.

The remainder of this article is organized as follows. Section II presents a brief overview of related work. Section III outlines the problem formulation. Section IV details the proposed method. Section V presents the experimental results. Finally, Section VI offers the conclusions.

II. RELATED WORK

In this section, we briefly review previous work: anomaly detection on attributed networks, graph representation learning, and contrastive learning. Finally, we provide a summary of comparisons between the proposed method and its related work.

A. Anomaly Detection on Attributed Networks

Detecting anomalies from attributed networks is of great importance as it has a vital function in mitigating risks. Attributed networks can be defined as graphs with attributed nodes and edges. So, the task is also referred to as GAD, which is different from other kind of tasks (e.g., time series [25] and videos [26]). Before the era of deep learning, traditional methods of shallow learning are often used for GAD. Perozzi and Akoglu [17] proposed a quality measure strategy for attributed neighborhoods that uses structure and attributes together to quantify both internal consistency and external separability. Li et al. [18] characterized the residuals of attribute reconstruction and their correlational features to detect anomalies. Peng et al. [19] improved GAD performance by jointly considering CUR decomposition (C, U, and R stand for the three matrices into which this decomposition can break a matrix.) and residual analysis results. The performance of these methods is limited by their shallow mechanisms.

Recently, deep neural networks have shown tremendous potential in mining latent patterns from high-dimensional graph data [27], [28], [29], [30], [31], [32], [33]. Contrastive learning method is frequently employed for deep GAD. In [8], the node–subgraph contrast method is first proposed to learn the representative information from the previously sampled node–subgraph instance pairs. This method has been widely used due to its simple and effective contrastive learning mechanism. To improve the feature representation, node–node and subgraph–subgraph contrasts are proposed by [11] and [23], respectively. Utilizing

a multiscale learning strategy can further explore the latent patterns to achieve better performance. The reconstruction strategy is another viable choice for GAD. In [9], an autoencoder framework is employed to reconstruct the original graph with the learned node embeddings from GNN. Anomalies are then flagged by reconstruction errors. Similarly, Zhang et al. [13] provided a masked autoencoder-based reconstruction module to identify the nodes with large reconstruction errors. In [34], attribute reconstruction based on generative learning schema is proposed to identify the distances between nodes and their neighbors in the attribute space. In contrast to the methods mentioned above, Ding et al. [35] used a generative adversarial training strategy to improve the generalization ability of the model on newly added data. In [36], anomalies are distinguished by a learned hypersphere with normal nodes clustered around the center.

B. Graph Representation Learning

GNN is a powerful tool for graph representation learning. Nodes from attributed networks are richly related to each other, which can be well captured by graph learning. Many different types of GNNs have been developed and used for graph representation, e.g., graph convolutional networks (GCNs) [37], graph attention network (GAT) [38], and GraphSage [39]. In [27], graph learning is combined with Transformer [40] to better represent the roles of nodes in an evolving graph space. Similarly, graph structure learning module [41] based on Gumbel-softmax sampling is integrated into the encoding blocks of the Transformer. To improve graph learning of normal patterns, hierarchical memory blocks are developed in [16] to record prototypical patterns of normal samples and prevent unexpected generalization to abnormal samples. In [42], graph representation learning is enhanced through local and global regularization techniques. In [43], a Haar-type graph framelet network is introduced for multiscale learning on heterophilous graphs. In [44], quantum walk kernels are introduced to better capture complex inherent features of graphs for classification purposes. In [45], an adaptive multifrequency GNN framework is designed to capture both low- and high-frequency information of neighboring nodes. This approach allows for the creation of more distinct representations. In general, GNNs are widely employed to model graph data and to learn the relationships between nodes.

C. Contrastive Learning

Contrastive learning aims to improve performance by matching representations under appropriate transformations [10]. A comprehensive overview of mainstream contrastive learning approaches in various domains is provided in [46]. Typically, data augmentation techniques are employed initially to create various views for learning. For time series or visual data, Gaussian noise and random/adaptive masking [47] are commonly utilized for data augmentation and view generation. However, the attributed networks are more complex due to the correlations

between the nodes. To deal with graph data, researchers have developed different augmentation strategies that target either node attributes or the structure of the graph. Ding et al. [48] presented a structured survey of graph data augmentation. You et al. [10] designed four types of graph data augmentations [node dropping, edge perturbation, attribute masking (AM), and subgraph]. The views generated by these strategies are then used to maximize mutual information. In [49], graph views are generated by an adaptive augmentation method that can preserve intrinsic structures and attributes. In contrast to the predefined view generation methods, adaptive augmentation strategies can provide more informative contrastive views, leading to better performance. In [50], graph views are generated by perturbing the node embeddings instead of introducing complex operations over graphs. Then, a neighbor-oriented contrastive loss is developed for training. In the GAD task, the views are usually generated by AM or subgraph extraction. Different contrastive learning methods (node–node [8], node–subgraph [11], and subgraph–subgraph [23]) have been developed to improve the performance. Unlike the above methods based on graph data augmentation, in [51], the GNN encoder is perturbed to generate an augmented branch for learning. Such methods can maintain the semantics of highly sensitive graph data while producing a qualified view for contrast. In general, augmentation is designed to generate different views for contrastive learning. The objective of contrastive learning is to develop an embedding space where similar sample pairs are positioned close together, while dissimilar pairs are located farther apart. With the limited training data, contrastive learning can maximize the utilization of training data to improve pattern representation and generalization capabilities.

In this article, we present MMGAD that uses node–node and node–subgraph contrasts as a foundation and utilizes GCL to improve representation learning. The parameter perturbation method [51] is employed to create views for cross-view contrast. We also provide a cycle training strategy to refine the model. To summarize, MMGAD differs from the related work [23] in three main aspects: 1) MMGAD utilizes a network view augmentation method based on parameter perturbation for cross-view GCL; 2) MMGAD performs additional graph-level GCL for both node–node and node–subgraph branches; and 3) MMGAD introduces a new cycle training strategy to improve performance.

III. PROBLEM FORMULATION

In this article, we focus on the problem of detecting anomalies from the attributed graph. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ be an attributed graph. It is an undirected graph with a node set $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ and an edge set $\mathcal{E}$. $\mathbf{X} \in \mathbb{R}^{n \times d}$ denotes the attribute matrix. There are n nodes, each of which has an attribute vector with length d . Thus, the attributes for the node v_i can be defined as $\mathbf{x}_i \in \mathbb{R}^d$, $1 \leq i \leq n$. The edge set $\mathcal{E}$ is based on an adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, which represents the underlying graph structure of $\mathcal{G}$. Specifically, $\mathbf{A}_{i,j} = 1$ indicates that there is an edge between nodes v_i and v_j , otherwise $\mathbf{A}_{i,j} = 0$.

TABLE I
SUMMARY OF THE NOTATIONS

Notations	Definitions
$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$	An undirected attributed graph
$\mathcal{V}$	The node set of $\mathcal{G}$
$\mathcal{E}$	The edge set of $\mathcal{G}$
$\mathbf{X} \in \mathbb{R}^{n \times d}$	The attribute matrix of $\mathcal{G}$
$\mathbf{A} \in \mathbb{R}^{n \times n}$	The adjacency matrix of $\mathcal{G}$
$\mathbf{x}_i \in \mathbb{R}^d$	The attribute vector for the i th node in $\mathcal{G}$
v_i	The i th node in $\mathcal{V}$
$\mathcal{G}_i$	The subgraph of the target node v_i
$\mathbf{X}_i \in \mathbb{R}^{m \times d}$	The attribute matrix of subgraph $\mathcal{G}_i$ with m nodes
$S : (\cdot)$	Learned anomaly score function for $\mathcal{G}$.
n	The number of nodes in $\mathcal{G}$
d	The dimension of node attributes in $\mathcal{G}$
m	The size of subgraph
$f(\cdot, \theta)$	The GNN module with weight parameters θ
$\mathbf{H}_{nn}^i, \hat{\mathbf{H}}_{nn}^i$	Graph representations of $\mathcal{G}_i$ at node–node level
$\mathbf{H}_{ns}^i, \hat{\mathbf{H}}_{ns}^i$	Graph representations of $\mathcal{G}_i$ at node–subgraph level
$\mathbf{h}_{nn}^i$	Node embedding of v_i by GNN at node–node level
$\mathbf{h}_{ns}^i$	Graph embedding of v_i by GNN at node–subgraph level
$\mathbf{e}_{nn}^i, \mathbf{e}_{ns}^i$	Node embeddings of v_i by MLPs

The goal of anomaly detection on attributed networks is to identify anomalous nodes in the graph. Due to the unsupervised manner, there is no ground truth indicating which nodes are positive. That is to say, the graph data mixed with anomalous nodes as rare categories are directly used for training any annotation. Generally, given an attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, the target is to learn a score function $S : (\cdot)$, which takes the graph $\mathcal{G}$ as input and assigns an anomaly score to each node in $\mathcal{G}$. A larger score indicates that the corresponding node is of higher abnormality. With a proper threshold, anomalous nodes can be flagged easily. For convenience, the main notations used in this article are summarized in Table I.

IV. METHODOLOGY

A. Overall Framework

The overall architecture of the proposed framework, MMGAD, is shown in Fig. 1. It is an unsupervised and multi-task framework that partially follows the mainstream workflow: sampling, contrastive learning, and scoring. In the sampling stage, target node v_i and its subgraph $\mathcal{G}_i$ are first generated. Then, they are fed into the proposed model for contrastive learning. The subgraphs around the target nodes are sampled using the popular random walk with restart (RWR) method [52], which has also been used in many previous works. The proposed model consists of two parallel branches for multiscale learning: node–node contrast and node–subgraph contrast. Each branch uses GNN-based networks to learn the latent patterns from the input subgraph data. To improve the representation and generalization capabilities, we provide a multiview GCL method as an auxiliary task for the multiscale learning task. Unlike existing methods that generate views by data augmentation, we directly augment the GNN modules by perturbing the training parameters to generate new views for graph-level contrastive learning. More specifically, a new view

is generated in each branch by perturbing the parameters of the corresponding GNN module. For example, the GNN module $f(\cdot, \theta_{ns})$ for node–subgraph contrast is perturbed to generate a new module defined as $f(\cdot, \theta'_{ns})$. We consider this as a new view from the perspective of networks (GNN function). With the same input data $\mathcal{G}_i$, the two views then output two representations, denoted as $\mathbf{H}_{ns}^i$ and $\hat{\mathbf{H}}_{ns}^i$. Finally, contrastive learning is used at the graph level to maximize the agreement of the two representations. In the scoring stage, the results from the two branches are combined by a weighted sum. We also develop a cycle training strategy that selects nodes with a high likelihood of being normal for an additional training step. This is an online sampling and training method that allows for continuous refinement of the model in an iterative process.

B. Multiscale Contrastive Learning

Multiscale contrastive learning is used to capture anomalous patterns at different scales. The task consists of two levels: node–node and node–subgraph contrasts. The node–node contrast mainly focuses on contextual anomalies, whereas the node–subgraph contrast captures the abnormal patterns of structural anomalies. The input for multiscale contrastive learning consists of the target nodes along with their subgraphs. To avoid information leakage, the target nodes within the corresponding subgraphs are masked using zero vectors. For contrast, we must first establish the positive and negative pairs. More precisely, a node and its subgraph form a positive pair. In contrast, a node and a subgraph of the other nodes form a negative pair.

1) *Node–Node Contrast*: Given a node v_i with attributes $\mathbf{x}_i$ and its subgraph $\mathcal{G}_i$, the goal of node–node contrast is to maximize the agreement between the embedding of v_i and the embedding of the corresponding masked node in $\mathcal{G}_i$. Specifically, the embedding of the masked node is aggregated from the other nodes of $\mathcal{G}_i$. First, a single-layer GNN module $f(\cdot, \theta_{nn})$ with weight parameters θ_{nn} is used to learn the representation $\mathbf{H}_{nn}^i$ for the subgraph $\mathcal{G}_i$

$$\begin{aligned} \mathbf{H}_{nn}^i &= f(\mathcal{G}_i, \theta_{nn}) \\ &= \sigma(\widetilde{\mathbf{D}}_i^{-\frac{1}{2}} \widetilde{\mathbf{A}}_i \widetilde{\mathbf{D}}_i^{-\frac{1}{2}} \mathbf{X}_i \theta_{nn}) \end{aligned} \quad (1)$$

where $\widetilde{\mathbf{A}}_i = \mathbf{A}_i + \mathbf{I}$ is the adjacency matrix with self-loop added, $\widetilde{\mathbf{D}}_i$ represents the degree of $\mathcal{G}_i$, $\mathbf{X}_i$ is the attribute matrix of $\mathcal{G}_i$, and $\sigma(\cdot)$ refers to the PReLU activation function. Although the target node in $\mathcal{G}_i$ is masked, the GNN module can easily aggregate messages for the target node from its neighbors. The target node embedding $\mathbf{h}_{nn}^i$ is selected from the learned subgraph representation $\mathbf{H}_{nn}^i$.

For contrast, we should also learn an embedding of the original node v_i . In addition, the node features should be mapped in the same hidden space. Therefore, the weight parameters should be shared with $\mathbf{h}_{nn}^i$. Given the attribute vector $\mathbf{x}_i$ of v_i , a multilayer perceptron (MLP) module is used to compute the node embedding $\mathbf{e}_{nn}^i$

$$\mathbf{e}_{nn}^i = \text{MLP}(\mathbf{x}_i) = \sigma(\mathbf{x}_i \theta_{nn}). \quad (2)$$

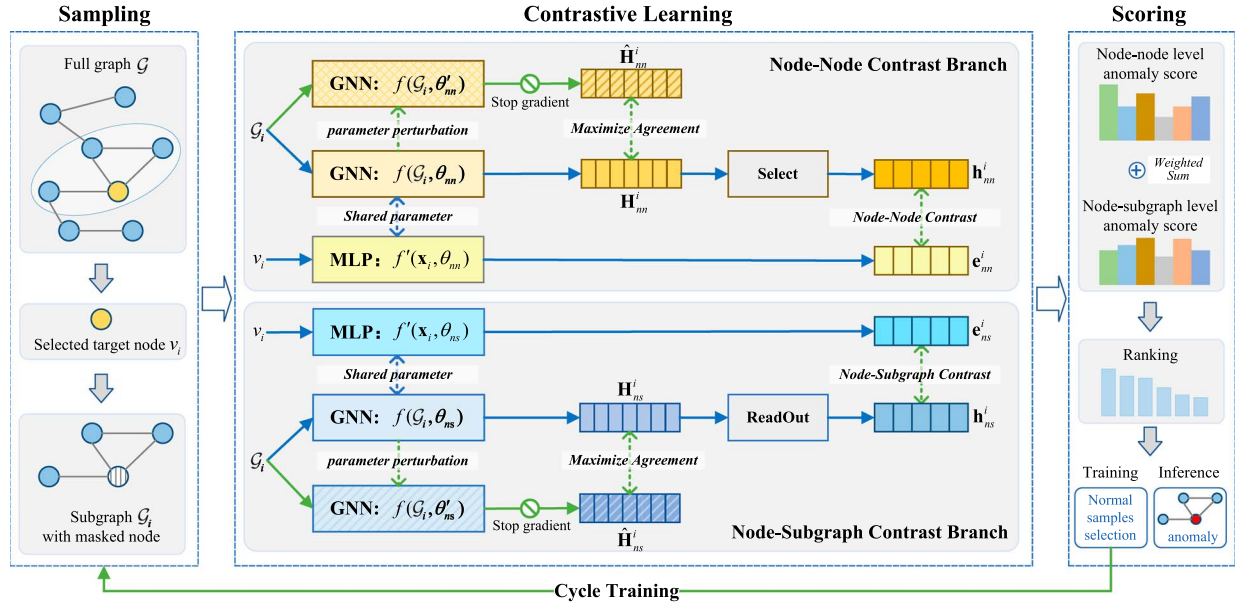


Fig. 1. Overview of the proposed MMGAD. It consists of three main components: sampling, contrastive learning, and scoring. In the sampling phase, target node v_i and its subgraph $\mathcal{G}_i$ are generated for training and inference. In the contrastive learning stage, there are two parallel branches for node–node contrast and node–subgraph contrast, respectively. In each branch, a new GNN view is generated by parameter perturbation-based network augmentation. Then, graph-level contrastive learning is used to maximize the agreement of the two views. For scoring, the results from two branches are merged for both cycle training and anomaly detection. It is worth noting that the processes associated with the green line are only activated in the training phase.

With $\mathbf{h}_{nn}^i$ and $\mathbf{e}_{nn}^i$, we can conduct contrast learning by maximizing the agreement. In detail, we use a bilinear layer to measure their similarity

$$\begin{aligned} s_{nn}^i &= \text{Bilinear}(\mathbf{h}_{nn}^i, \mathbf{e}_{nn}^i) \\ &= \text{Sigmoid}(\mathbf{h}_{nn}^i \mathbf{W}_{nn} (\mathbf{e}_{nn}^i)^\top) \end{aligned} \quad (3)$$

where $\mathbf{W}_{nn}$ is the weight matrix of the discriminator for node–node contrast. The aim of contrasting is to make the similarities of positive pairs tend toward 1 and negative pairs toward 0. Therefore, cross-entropy loss is used for contrastive learning

$$\mathcal{L}_{nn} = - \sum_{i=1}^n (y_i \log(s_{nn}^i) + (1 - y_i) \log(1 - s_{nn}^i)) \quad (4)$$

where the ground truth label y_i is set to 1 for positive pairs and 0 for negative pairs.

2) *Node–Subgraph Contrast*: Given the same node v_i with attributes $\mathbf{x}_i$ and its subgraph $\mathcal{G}_i$, node–subgraph contrast aims to maximize the agreement between the embedding of v_i and the embedding of $\mathcal{G}_i$. Similarly, a single-layer GNN module $f(\cdot, \theta_{ns})$ with weight parameters θ_{ns} is used to learn the representation $\mathbf{H}_{ns}^i$ for the subgraph $\mathcal{G}_i$

$$\begin{aligned} \mathbf{H}_{ns}^i &= f(\mathcal{G}_i, \theta_{ns}) \\ &= \sigma(\widetilde{\mathbf{D}}_i^{-\frac{1}{2}} \widetilde{\mathbf{A}}_i \widetilde{\mathbf{D}}_i^{-\frac{1}{2}} \mathbf{X}_i \theta_{ns}). \end{aligned} \quad (5)$$

Here, the node–subgraph contrast shares the same input with the node–node contrast. However, they are two independent and parallel branches. To learn the embedding of node v_i , we also

need to define a new MLP module. Similarly, the node features are mapped in the same hidden space as the subgraph features. The node embedding $\mathbf{e}_{ns}^i$ can be calculated as follows:

$$\mathbf{e}_{ns}^i = \text{MLP}(\mathbf{x}_i) = \sigma(\mathbf{x}_i \theta_{ns}). \quad (6)$$

Before contrast, a *Readout* module is defined to further transform the learned representation $\mathbf{H}_{ns}^i$ into a new embedding vector $\mathbf{h}_{ns}^i$. Specifically, the average pool function is employed as the *Readout* function. It is defined as follows:

$$\mathbf{h}_{ns}^i = \text{Readout}(\mathbf{H}_{ns}^i) = \sum_{j=0}^{n_i-1} \frac{\mathbf{H}_{ns}^i[j, :]}{n_i} \quad (7)$$

where n_i is the number of the nodes in the subgraph $\mathcal{G}_i$, and $\mathbf{H}_{ns}^i[j, :]$ is the j th row of the matrix $\mathbf{H}_{ns}^i$. In this way, the node embedding $\mathbf{e}_{ns}^i$ and the subgraph embedding $\mathbf{h}_{ns}^i$ have the same shape. Then, the bilinear layer is used again to measure their similarity

$$\begin{aligned} s_{ns}^i &= \text{Bilinear}(\mathbf{h}_{ns}^i, \mathbf{e}_{ns}^i) \\ &= \text{Sigmoid}(\mathbf{h}_{ns}^i \mathbf{W}_{ns} (\mathbf{e}_{ns}^i)^\top). \end{aligned} \quad (8)$$

Here, $\mathbf{W}_{ns}$ is the weight matrix of the discriminator for node–subgraph contrast. Likewise, the objective function of node–subgraph contrasting learning is defined as

$$\mathcal{L}_{ns} = - \sum_{i=1}^n (y'_i \log(s_{ns}^i) + (1 - y'_i) \log(1 - s_{ns}^i)) \quad (9)$$

where the ground truth label y'_i is set to 1 for positive pairs and 0 for negative pairs.

C. Multiview Contrastive Learning

The two GNN modules for graph learning defined above are key members of the multiscale contrast. To improve the representation of graph learning, multiview contrastive learning methods are often used. In general, a new view should first be generated by graph augmentation. The principle is to learn representations that are robust to data transformations [51]. However, the semantic patterns may be violated by the data augmentation. Although many impressive strategies have been proposed to mitigate the problem, such as adaptive augmentation [47] and the use of domain knowledge [53], the efficiency and flexibility are still limited.

Unlike the existing methods that directly augment the graph data, we simply perturb the GNN modules for multiview contrastive learning. The GNN modules are augmented by parameter perturbation to generate new GNN views. Specifically, we perturb the GNN modules $f(\cdot, \theta_{nn})$ and $f(\cdot, \theta_{ns})$ to generate $f(\cdot, \theta'_{nn})$ and $f(\cdot, \theta'_{ns})$, respectively. The functions of perturbation are defined as

$$\theta'_{nn} = \theta_{nn} + \lambda \cdot \Delta\theta_{nn}, \quad \Delta\theta_{nn} \sim \mathcal{N}(0, \sigma_{nn}^2) \quad (10)$$

$$\theta'_{ns} = \theta_{ns} + \lambda \cdot \Delta\theta_{ns}, \quad \Delta\theta_{ns} \sim \mathcal{N}(0, \sigma_{ns}^2). \quad (11)$$

Here, λ is the coefficient that scales the magnitudes of the two perturbations, and $\Delta\theta_{nn}$ and $\Delta\theta_{ns}$ are randomly sampled from Gaussian distributions with zero means and variances σ_{nn}^2 and σ_{ns}^2 , respectively. With the perturbed modules, we can easily produce new learned representations. For node–node contrast branch, a new learned representation $\hat{\mathbf{H}}_{nn}^i$ is calculated by the augmented GNN view $f(\cdot, \theta'_{nn})$. Similarly, $\hat{\mathbf{H}}_{ns}^i$ is computed by the augmented GNN view $f(\cdot, \theta'_{ns})$. It is worth noting that the GNN module and its augmented view share the same input graph data $\mathcal{G}_i$. The GNN functions are defined as

$$\hat{\mathbf{H}}_{nn}^i = f(\mathcal{G}_i, \theta'_{nn}) \quad (12)$$

$$\hat{\mathbf{H}}_{ns}^i = f(\mathcal{G}_i, \theta'_{ns}). \quad (13)$$

Then, we use the normalized temperature-scaled cross-entropy loss for multiview GCL. The loss is designed to enforce the agreement between positive pairs compared to negative pairs. Inspired by [51], we use graph-level contrastive learning to improve context expression. To be specific, with batch representations $\mathbf{H} = \{\mathbf{H}^0, \dots, \mathbf{H}^{b-1}\}$ and $\hat{\mathbf{H}} = \{\hat{\mathbf{H}}^0, \dots, \hat{\mathbf{H}}^{b-1}\}$, $(\mathbf{H}^i, \hat{\mathbf{H}}^i)$ is a positive pair and $(\mathbf{H}^i, \mathbf{H}^j)$ is a negative pair. Here, b is the batch size for training and $i \in [0, b-1]$, $j \in [0, b-1]$, $i \neq j$. The basic loss function $\mathcal{L}_{cl}$ is defined as

$$\mathcal{L}_{clb}(\mathbf{H}^i, \hat{\mathbf{H}}^i) = -\log \frac{\exp(\text{sim}(\mathbf{H}^i, \hat{\mathbf{H}}^i)/\tau)}{\sum_{j=0, j \neq i}^{b-1} \exp(\text{sim}(\mathbf{H}^i, \mathbf{H}^j)/\tau)} \quad (14)$$

$$\mathcal{L}_{cl}(\mathbf{H}, \hat{\mathbf{H}}) = \frac{1}{b} \sum_{i=0}^{b-1} \mathcal{L}_{clb}(\mathbf{H}^i, \hat{\mathbf{H}}^i). \quad (15)$$

Here, τ is the temperature parameter and $\text{sim}(\cdot, \cdot)$ is a cosine similarity function defined as follows:

$$\text{sim}(\mathbf{H}^i, \mathbf{H}^j) = \frac{\mathbf{H}^i \cdot \mathbf{H}^j}{\|\mathbf{H}^i\| \|\mathbf{H}^j\|}. \quad (16)$$

Finally, we can compute the cross-view contrastive loss at the graph level for node–node and node–subgraph branches. The total loss $\mathcal{L}_{mv}$ for multiview is obtained by adding the two cross-view losses

$$\mathcal{L}_{mv} = \mathcal{L}_{cl}(\mathbf{H}_{nn}, \hat{\mathbf{H}}_{nn}) + \mathcal{L}_{cl}(\mathbf{H}_{ns}, \hat{\mathbf{H}}_{ns}) \quad (17)$$

where $\mathbf{H}_{nn}$, $\hat{\mathbf{H}}_{nn}$, $\mathbf{H}_{ns}$, and $\hat{\mathbf{H}}_{ns}$ are batch features. For example, $\mathbf{H}_{nn} = \{\mathbf{H}_{nn}^0, \dots, \mathbf{H}_{nn}^{b-1}\}$.

According to [54] and [55], alignment and uniformity are two key metrics to measure the quality of contrastive learning. When the positive pairs are sampled from the data space, the two metrics can be defined as follows:

$$\mathcal{L}_{align}(f, \theta) \triangleq \mathbb{E}_{(x,y) \sim p_{pos}} [\|f(x, \theta) - f(y, \theta)\|_2^2] \quad (18)$$

$$\mathcal{L}_{uniform}(f, \theta) \triangleq \log \mathbb{E}_{x,y \stackrel{i,i,d}{\sim} p_{data}} [e^{-2\|f(x, \theta) - f(y, \theta)\|_2^2}] \quad (19)$$

where p_{pos} denotes the positive pairs and p_{data} refers to the data distribution. $\mathcal{L}_{align}$ is designed to measure the distance between the positive samples in the hidden space. $\mathcal{L}_{uniform}$ is employed to indicate whether the random samples are uniformly distributed on the hypersphere of the hidden space. Traditional data augmentation methods can generate different types of samples for training. By using the basic loss function $\mathcal{L}_{cl}(\mathbf{H}^x, \mathbf{H}^y)$ as the learning objective, $\mathcal{L}_{align}$ and $\mathcal{L}_{uniform}$ can be improved (lower is better), resulting in better representation capability.

However, due to the high sensitivity of graph structures, augmentation from the data perspective may violate the latent patterns. Moreover, it is difficult and tedious to find out the optimal data augmentation strategies. We employ the parameter perturbation method for contrastive learning to avoid tampering with the original data. The augmented views are generated by the perturbed module. In this way, the metric $\mathcal{L}_{align}$ can be defined as follows:

$$\mathcal{L}_{align}(f, \theta, \theta') \triangleq \mathbb{E}_{x \sim p_{data}} [\|f(x, \theta) - f(x, \theta')\|_2^2]. \quad (20)$$

Here θ' refers to the perturbed parameters of f . Similarly, the loss function $\mathcal{L}_{cl}(\mathbf{H}^x, \hat{\mathbf{H}}^x)$ can also minimize $\mathcal{L}_{align}$ to achieve better representations. For $\mathcal{L}_{uniform}$, there is no difference in the description of the formula. Since the perturbations of the parameters are only sampled from Gaussian distributions with very small magnitudes, the inherent patterns in the embeddings can be preserved. Learning with $\mathcal{L}_{cl}$, $\mathcal{L}_{uniform}$ can also be improved well. To summarize, the parameter perturbation method is theoretically effective for contrastive learning.

Our multiview contrastive learning method differs from the existing parameter perturbation method [51] in three main aspects: 1) our method combines both node–node and node–edge contrastive learning for the task of anomaly detection; 2) we employ a cycle training approach to enhance the representation capabilities, whereas [51] uses adversarial training to improve them; and 3) for the parameter perturbation-based contrast part, our method builds on [51] by adapting the parameters specifically for the anomaly detection task and running them in parallel on both the node and edge branches.

D. Scoring and Training

1) *Scoring*: We use a similar scoring method as in [8], [11], and [23]. In particular, positive pairs and negative pairs are included in the sample in equal numbers. Since the similarities of the positive pairs tend to 1 and those of the negative pairs tend to 0, the score for the node–node contrast can be defined as follows:

$$S_{nn}(v_i) = s_{nn}^i(-) - s_{nn}^i(+) \quad (21)$$

where $s_{nn}^i(-)$ and $s_{nn}^i(+)$ are the similarities for negative and positive pairs of the node–node contrast, respectively. Similarly, the score for the node–subgraph contrast is as follows:

$$S_{ns}(v_i) = s_{ns}^i(-) - s_{ns}^i(+). \quad (22)$$

The fused score is a weighted sum of the node–node and node–subgraph scores

$$S_{\text{sum}}(v_i) = \alpha S_{nn}(v_i) + (1 - \alpha) S_{ns}(v_i) \quad (23)$$

where α is a tradeoff factor. According to the previous works [8], [11], [23], multiround scoring should be performed to calculate the anomaly score for each node. The final score $S(v_i)$ for node v_i is calculated by the mean and standard deviation

$$S(v_i) = \bar{S}(v_i) + \sqrt{\frac{1}{R} \sum_{r=0}^{R-1} (S_r(v_i) - \bar{S}(v_i))^2}. \quad (24)$$

Here, R is the number of scoring rounds, $S_r(v_i)$ is the score of the r th round, and $\bar{S}(v_i) = (1/R) \sum_{r=0}^{R-1} S_r(v_i)$ is the mean of the R rounds scores.

2) *Cycle Training*: In the training phase, the multiscale learning task and multiview GCL task are jointly trained. For the multiscale learning task, the objective function is as follows:

$$\mathcal{L}_{\text{ms}} = \alpha \mathcal{L}_{nn} + (1 - \alpha) \mathcal{L}_{ns} \quad (25)$$

where α is shared with (23). Finally, the overall objective function is defined as

$$\mathcal{L} = \mathcal{L}_{\text{ms}} + \beta \mathcal{L}_{\text{mv}} \quad (26)$$

where β is a weight factor to balance the two losses.

With $\mathcal{L}$, we can learn the model jointly. However, anomalous nodes in the training data may affect the learning of normal patterns and weaken the discrimination capability. To solve this problem, we develop an online cycle training strategy as shown in Fig. 2. Specifically, we first learn the model for a certain rate (denoted as p) of the total steps through normal training so that the model has a basic discrimination capability. Then, we continue to learn the model by performing normal training and cycle training simultaneously. The normal training is performed and followed by a cycle training process that forms a new step training.

The cycle training process consists of several operations. First, we calculate the anomaly scores of the nodes in the graph using the current model. Then, the nodes are ordered by their scores to select nodes with low-anomaly scores. The selected rate is defined as q . After selection, most of the anomalous

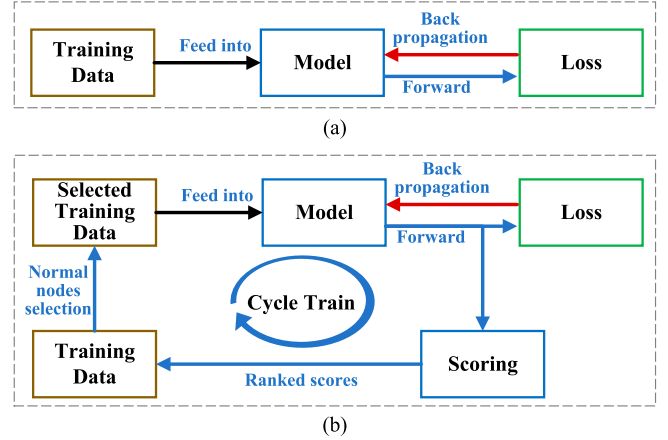


Fig. 2. Training process. (a) Normal training process. (b) Proposed cycle training strategy that selects normal nodes as new training data for online model learning.

nodes are removed from the training data. Finally, the selected training data are fed into the model for an additional training step.

E. Complexity Analysis

There are three main stages that might affect the time complexity of MMGAD: subgraph sampling, contrastive learning, and scoring. In the inference phase, the time complexity of subgraph sampling is determined by the RWR method. Given the average node degree of the graph ε , the number of nodes n , and the size of the subgraph m , the complexity of sampling is $\mathcal{O}(m n \varepsilon)$. The time complexity of contrastive learning is mainly influenced by the GNN modules. The time complexity can be defined as $\mathcal{O}(2m^2)$ for each node and $\mathcal{O}(2m^2 n)$ in total. For scoring, the computational time complexity is very low and can be considered negligible. Considering R rounds of detection, the overall time complexity is $\mathcal{O}(m n R (2m + \varepsilon))$. In the training phase, the computation of the loss function and gradient back propagation are required. The time complexity of contrastive learning is doubled by the additional perturbed GNN modules. The cost of parameter perturbation and normal nodes selection can be neglected due to their constant complexity.

V. EXPERIMENTS

In this section, we conduct extensive experiments on six widely used benchmark datasets to prove the effectiveness of our proposed MMGAD framework. Performance comparison with the state-of-the-art methods, ablation study, sensitivity analysis and limitations are presented in detail.

A. Datasets

The six benchmark datasets used in the experiments are listed in Table II. BlogCatalog and Flickr [56] are social network datasets collected from the blog website BlogCatalog and the image website Flickr, respectively. Cora [57], CiteSeer [57], and ACM [59] are citation graph datasets collected from the

TABLE II
STATISTICS OF THE DATASETS

Datasets	Nodes	Edges	Attributes	Anomalies
BlogCatalog [56]	5196	171 743	8189	300
Flickr [56]	7575	239 738	12 407	450
Cora [57]	2708	5429	1433	150
CiteSeer [57]	3327	4732	3703	150
UAI2010 [58]	3067	28 311	4973	150
ACM [59]	16 484	71 980	8337	600

corresponding publication databases. UAI2010 [58] is a community detection dataset. Following previous works [8] and [9], we inject contextual and structural anomalies into the original graph datasets. For contextual anomalies, first a target node v_i and a candidate set with another 50 nodes are randomly selected. Then, the attributes of v_i are replaced by the attributes of the most dissimilar node from the set. For structural anomalies, 15 nodes are randomly selected from the graph and fully connected. Finally, we repeat the operations and inject the same number of contextual and structural anomalies into the graphs. The total number of anomalies is controlled according to the size of each dataset.

B. Experimental Setup

1) *Baselines*: We compare MMGAD with eight state-of-the-art methods for GAD, given as follows.

- a) Radar [18]: Radar models the residuals of node attributes and their coherence with network information for GAD.
- b) ANOMALOUS [19]: It is a joint learning framework that selects attributes and detects anomalies based on CUR decomposition and residuals.
- c) DOMINANT [9]: It is a reconstruction-based method that measures anomalies based on the reconstruction errors.
- d) CoLA [8]: It is a contrastive unsupervised method that detects anomalies based on node–subgraph contrast.
- e) ANEMONE [11]: It is a multiscale method that uses both node–node and node–subgraph contrasts.
- f) SL-GAD [34]: It is a generative and contrastive framework that uses generative attribute regression and multi-view contrastive learning for anomaly detection.
- g) GRADATE [23]: It is a multiview method that adds subgraph–subgraph contrast based on augmented view to improve performance.
- h) NLGAD [15]: It is a two-stage framework consisting of normality selection and multilevel contrastive learning.

2) *Metrics*: To measure the performance and make a fair comparison, we choose ROC-AUC as the metric. ROC-AUC is also frequently used in the earlier works. The ROC curve shows the performance of a binary classifier with different decision thresholds. It plots the true-positive rate (TPR) against the false-positive rate (FPR). The AUC value is the area under the ROC curve. It summarizes how well a model can produce relative values that differentiate between positive and negative samples across all detection thresholds. The AUC value ranges from 0 to 1. A higher AUC value signifies better performance.

3) *Settings*: To be efficient, the size of the subgraphs is set to 4. For each branch, only one GNN layer is stacked for graph learning. The hidden dimension of the GNN modules is set to 64. α is chosen from different settings $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$, and β is chosen from $\beta \in \{1, 0.1, 0.01, 0.001, 0.0001\}$. The coefficient of the parameter perturbation λ is chosen from the set $\lambda \in \{0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5\}$. The normal training rate p is chosen from $p \in \{0, 0.25, 0.5, 0.75, 1\}$ to perform normal training for the first $p \times 100$ percentage of the total steps. The selection rate q is chosen from the set $q \in \{0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$, which indicates the percentage of normal nodes selection. The Adam optimizer is employed with an initial learning rate of 0.002 for BlogCatalog, 0.001 for Flickr, Cora, CiteSeer, UAI2010, and ACM. The number of epochs is set to 600 for BlogCatalog and Flickr, 200 for Cora, 100 for CiteSeer and ACM, and 300 for UAI2010. The round number R for scoring is set to 256. Our model is implemented with PyTorch [60] and trained end-to-end. All experiments are performed on a laptop with 32 GB dedicated memory, an Intel Core i7 CPU, and an NVIDIA GeForce RTX 3080 GPU. Our code is available at <https://github.com/shuxin-qin/MMGAD>.

C. Performance Comparison

We compare MMGAD with the eight baseline methods mentioned above. The results of the AUC values are shown in Table III, and the comparison of ROC curves is shown in Fig. 3. According to the comparison, we can make the following observations.

- 1) The proposed MMGAD outperforms all the baseline methods on the six datasets. It yields 0.08%, 0.09%, 0.42%, 0.46%, 0.49%, and 1.44% absolute AUC value promotions compared with the second-best values on the six datasets, respectively. The average AUC gain is 1.05%. The results demonstrate the effectiveness of our proposed framework.
- 2) In general, deep learning methods achieve better results than shallow methods (Radar and ANOMALOUS), whose performance is limited by the hand-crafted feature extraction and shallow learning capabilities. Deep graph learning is a better choice to capture anomalous patterns from graphs with complex structure and high-dimensional features.
- 3) Based on graph learning, anomaly detection methods such as reconstruction [9], [34], multilevel contrast [11], multiview contrast [23], and normality learning [15] have been proposed. They have improved the performance to a certain extent. Specifically, DONIMANT and SL-GAD are based on attribute reconstruction. A decoder is required for the reconstruction and anomaly ranking, which incurs additional computational costs. By adding contrastive learning, SL-GAD achieves better results than DONIMANT. Moreover, SL-GAD achieves the second-best results on BlogCatalog and Flickr, both of which contain more edges. We argue that the reconstruction-based methods easily suffer from overfitting on small datasets. Multilevel contrast has been shown to be an

TABLE III
PERFORMANCE COMPARISON WITH BASELINES ON AUC VALUES

Methods		BlogCatalog	Flickr	Cora	CiteSeer	UAI2010	ACM	Average
Radar [18]		0.7401	0.7399	0.6587	0.6709	0.7132	0.7047	0.7046
ANOMALOUS [19]		0.7237	0.7434	0.5770	0.6307	0.7038	0.7201	0.6831
DONIMANT [9]		0.7468	0.7442	0.8155	0.8251	0.7736	0.7601	0.7776
CoLA [8]		0.7854	0.7513	0.8779	0.8968	0.8354	0.8237	0.8284
ANEMONE [11]		0.7992	0.7712	0.9057	0.9189	0.8913	0.8462	0.8554
SL-GAD [34]		0.8184	0.7966	0.9130	0.9136	0.8729	0.8538	0.8614
GRADATE [23]		0.8037	0.7796	0.9237	0.9311	0.9262	0.8546	0.8698
NLGAD [15]		0.8012	0.7807	0.9286	0.9457	0.9320	0.8655	0.8756
MMGAD		0.8192	0.7975	0.9328	0.9503	0.9369	0.8799	0.8861

Note: The best values are highlighted in bold and the second best are underlined.

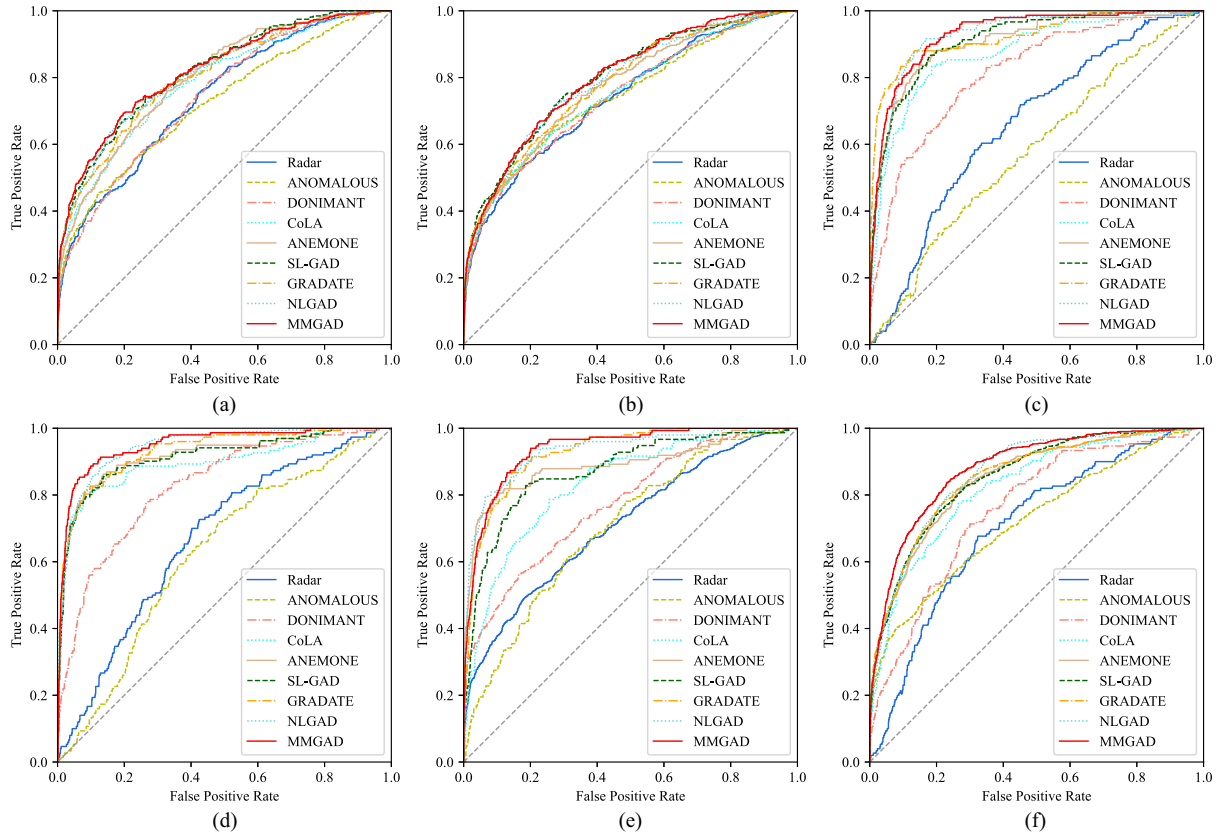


Fig. 3. ROC curves comparison on six benchmark datasets: (a) BlogCatalog; (b) Flickr; (c) Cora; (d) CiteSeer; (e) UAI2010; and (f) ACM. The larger the area under the curve, the better the anomaly detection performance.

effective strategy to improve anomaly detection performance. ANEMONE, GRADATE, NLGAD, and MMGAD are all based on the multilevel strategy. In detail, ANEMONE uses only nodes and subgraphs for multi-level contrast. GRADATE provides subgraph-subgraph contrast as a complement by using edge modification (EM) to create a new view. In this way, the quality of feature representations can be improved. However, direct graph augmentation may limit the improvement due to the high sensitivity of the original networks. In contrast, NLGAD designs a normality learning module that selects

normal nodes for contrastive learning at each step. This is an effective method to strengthen the discrimination capability. As a result, it achieves the second-best performance on four datasets except BlogCatalog and Flickr. Accordingly, we argue that there is still room for further improvement in representation and discrimination capabilities, especially on large datasets.

- 4) After a comprehensive comparison, we propose a cross-view GCL method based on network augmentation and cycle training strategy to avoid modifying the sensitive training data and improve the learning of normal

TABLE IV
ABLATION STUDY OF MULTIVIEW GCL W.R.T. AUC

Methods	BlogCatalog	Flickr	Cora	CiteSeer	UAI2010	ACM
<i>w/o MV</i>	0.8002	0.7758	0.9175	0.9214	0.9021	0.8522
<i>with MV_{nn}</i>	0.7998	0.7839	0.9246	0.9318	0.9163	0.8540
<i>with MV_{ns}</i>	0.8127	0.7918	0.9306	0.9462	0.9298	0.8648
MMGAD	0.8192	0.7975	0.9328	0.9503	0.9369	0.8799

Note: The best values are highlighted in bold.

patterns, respectively. Our method utilizes the advantages of multiview GCL and normality learning and compensates for the weaknesses of the baseline methods. First, network augmentation and graph-level contrastive learning force the node–node and node–subgraph branches to become more representative and generalized. Second, the proposed cycle training can further strengthen the discrimination capability by training with collected normal samples for additional steps. Our cycle training is different from NLGAD in two aspects: 1) the cycle training is fully integrated into the training phase in a circular manner, while NLGAD is a two-phase framework consisting of normality selection and normality learning; and 2) we provide a simple but efficient selection strategy which is different from NLGAD. As a result, our method is more efficient. For example, NLGAD requires 700 epochs of total training on Cora, while MMGAD requires only 200 epochs with cycle training (which is equivalent to 300 epochs since we perform cycle training for the last half of the total steps).

D. Ablation Study

In this section, we conduct an ablation study to clearly verify the effectiveness of the main components of MGCLAD, including the multiview GCL, the parameter perturbation-based augmentation, and the cycle training strategy.

1) *Effectiveness of Multiview GCL*: We add multiview GCL for both node–node and node–subgraph branches to improve graph representation. To verify the effectiveness, we run the experiments with three different configurations: without multiview GCL (denoted as *w/o MV*), only with multiview for node–node branch (denoted as *with MV_{nn}*), and only with multiview for node–subgraph branch (denoted as *with MV_{ns}*). As shown in Table IV, the proposed multiview GCL method achieves significant AUC improvements on the six datasets for both node–node and node–subgraph branches. Moreover, using multiview GCL for a single branch can also improve the performance. In general, the proposed multiview GCL is a cross-view and graph-level contrastive learning method that can improve the graph representations for both node–node and node–subgraph contrasts. With better graph representations, MMGAD achieves higher AUC values.

2) *Effectiveness of Parameter Perturbation*: We use a network augmentation method based on parameter perturbation to generate new views for GCL. Existing methods usually use

TABLE V
EFFECTIVENESS OF PARAMETER PERTURBATION ON AUC VALUES

Methods	BlogCatalog	Flickr	Cora	CiteSeer	UAI2010	ACM
<i>using AM</i>	0.8006	0.7787	0.9232	0.9267	0.9184	0.8492
<i>using EM</i>	0.8079	0.7814	0.9218	0.9381	0.9175	0.8617
MMGAD	0.8192	0.7975	0.9328	0.9503	0.9369	0.8799

Note: The best values are highlighted in bold.

graph augmentation to generate views. Our method directly augments the GNN networks to form two parallel GNNs with the same input graph data, while the other methods put the input graph data and its augmented version into the same GNN networks. To verify the effectiveness of parameter perturbation, commonly used graph augmentation methods are employed for comparison, including AM and EM. AM is a contextual augmentation method in which parts of the node attributes are randomly masked. EM is a structural modification strategy in which edges are randomly removed and added. As shown in Table V, *using AM* and *using EM* indicate that our parameter perturbation-based network augmentation mechanism is replaced by AM and EM, respectively. The proposed method achieves better performance than the traditional graph augmentation methods. We argue that the modification of graph data may destroy the inherent semantics due to the high sensitivity of graph data, which limits the performance. The key of GCL is to improve the alignment and uniformity of graph embeddings through contrastive learning. With the perturbed GNN branches, our method can not only meet the requirements of GCL naturally but also preserve the data semantics of the graph.

We also provide a visualization of the learning results based on the Cora dataset. It is shown in Fig. 4, where each point represents a node with its node–node and node–subgraph scores as 2-D coordinate values. The normal nodes are blue, while the abnormal nodes are red. For a better view, only the first 1000 nodes are visualized. There are four subfigures representing the learning results under different settings: without multiview GCL, using AM for graph augmentation, using EM for graph augmentation, and the proposed MMGAD. The point with higher node–node score and node–subgraph score indicates a higher probability of being an abnormal node. Based on Fig. 4, we can make the following observations: 1) most of the abnormal nodes are located in the upper right area of each subfigure. However, there are also some red points scattered in the low-score areas, which leads to an increase in the number of false negatives. 2) MMGAD achieves a better learning result than the other three settings [Fig. 4(d)]. The parameter perturbation-based GCL forces the node–node and node–subgraph branches to become more representative and generalized. As a result, the node–node and node–subgraph scores become more discriminative. 3) GCL methods based on traditional graph augmentation (AM and EM) fail to achieve significant improvement [Fig. 4(b) and 4(c)]. Random data augmentation on highly sensitive graph networks is uncontrollable and uncertain, leading to a limited improvement in representation and generalization capabilities.

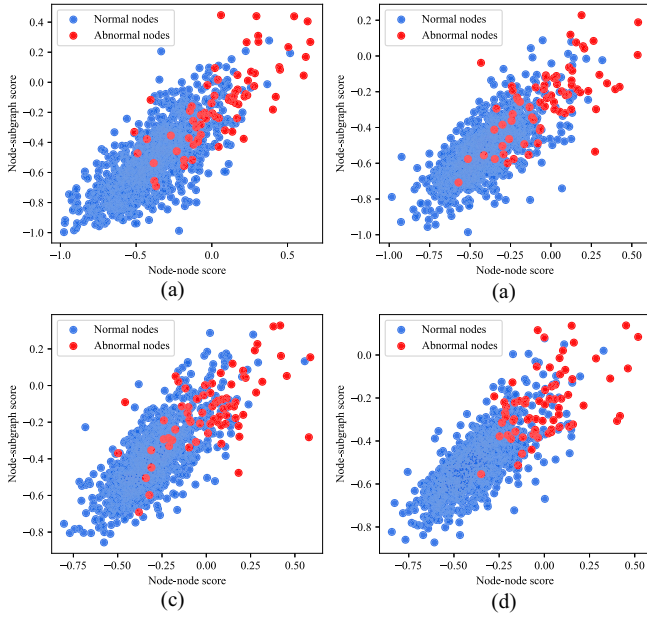


Fig. 4. Visualization of learning results under different settings using the Cora dataset: (a) without multiview; (b) using AM; (c) using EM; and (d) MMGAD. We use the node–node score and node–subgraph score as the 2-D coordinate to draw the corresponding node. The normal nodes are blue, while the abnormal nodes are red. For a better view, only the first 1000 nodes are visualized.

TABLE VI
ABLATION STUDY OF CYCLE TRAINING W.R.T. AUC

Methods	BlogCatalog	Flickr	Cora	CiteSeer	UAI2010	ACM
<i>w/o</i>	0.8108	0.7931	0.9247	0.9356	0.9289	0.8587
$p = 1$	0.8127	0.7914	0.9255	0.9428	0.9326	0.8691
$p = 0.5$	0.8192	0.7975	0.9328	0.9503	0.9369	0.8799

Note: The best values are highlighted in bold.

3) *Effectiveness of Cycle Training*: We design a cycle training strategy to improve the model’s learning of normal patterns and discrimination ability. We compare the performance with three different settings: without cycle training (denoted as *w/o*), with cycle training starting from the first step ($p = 1$), and with cycle training for the last half of the total steps ($p = 0.5$). As shown in Table VI, cycle training can significantly improve the performance of these datasets. Using cycle training from the first step is not the best choice, because the selection results have low credibility at the beginning of training. Therefore, we set $p = 0.5$ in our experiments, considering the performance and training time. We also provide the plots of the anomaly score distributions with and without cycle training on Cora and ACM datasets, as shown in Fig. 5. We first train models on both datasets without the cycle training strategy. Then, the cycle training is used in the training phase for comparison. From the visualization results, the anomaly scores generated by the models with cycle training are more discriminative than those generated by the model without cycle training. Theoretically, anomalies are the nodes that deviate from the normal patterns.

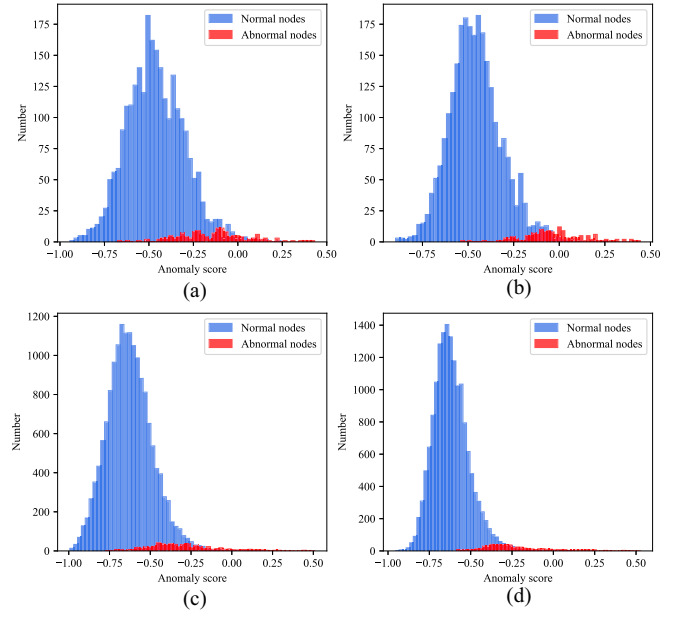


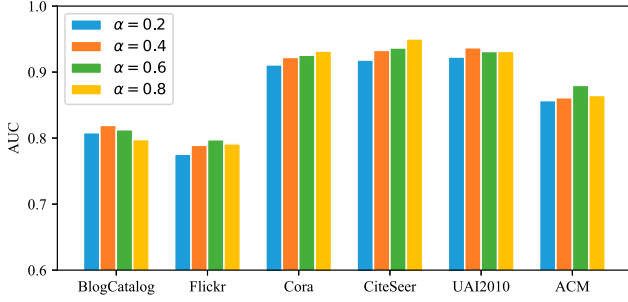
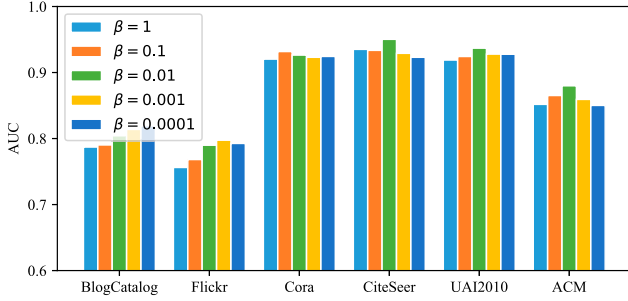
Fig. 5. Diagrams of the distribution of anomaly scores. (a) and (b) Results of models training on Cora. (c) and (d) Results training on ACM. Without cycle training means that the model is trained without the proposed cycle training strategy, while with cycle training it means the proposed MMGAD. The overlapping areas indicate the discrimination capability of the corresponding model. (a) Without cycle training on Cora. (b) With cycle training on Cora. (c) Without cycle training on ACM. (d) With cycle training on ACM.

Under the unsupervised paradigm, abnormal nodes mixed into the training data may interfere with the learning of normal patterns and thus weaken the discrimination capability to some extent. To mitigate this problem, nodes with lower anomaly scores (considered as normal nodes) are collected online for an additional training step. In this way, the learning of normal patterns can be improved. The experimental results mentioned above have confirmed our analysis.

E. Sensitivity Analysis

In this subsection, we conduct several experiments to investigate the effects of the main hyperparameters on the performance of MMGAD, including the weight factors α and β , the coefficient of parameter perturbation λ , the normal training rate p , the selection rate q , the subgraph size, and the hidden dimension of the GNN modules.

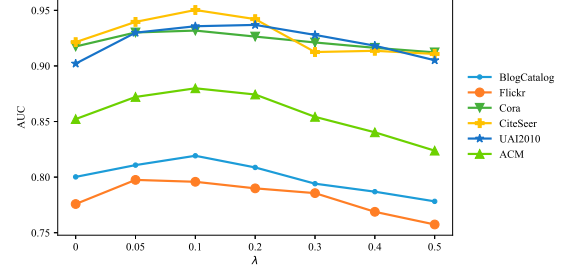
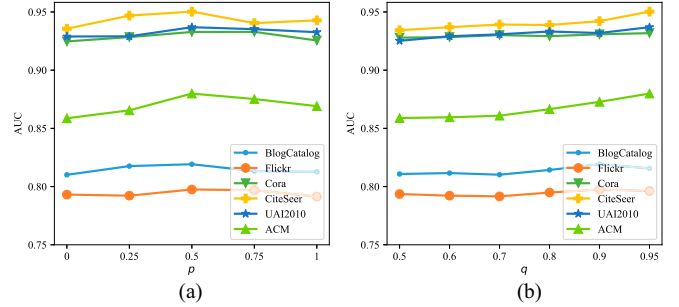
1) *Weight Factors α and β* : The weight factor α is used to balance the node–node and node–subgraph branches in both the training and inference phases. Fig. 6 shows the performance of MMGAD under different settings of $\alpha \in \{0.2, 0.4, 0.6, 0.8\}$. For different datasets, the settings of α that lead to the best performances are also different. The best settings for BlogCatalog, Flickr, Cora, CiteSeer, UAI2010, and ACM are 0.4, 0.6, 0.8, 0.8, 0.4, and 0.6, respectively. The weight factor β is used to balance the two learning tasks: multiscale learning and multiview GCL. It is only used in the training phase. As shown in Fig. 7, the AUC values change with different settings of β . The best settings of β for these datasets are 0.0001, 0.001, 0.1, 0.01, 0.01, and 0.01,

Fig. 6. AUC values of MMGAD with different weight factor α .Fig. 7. AUC values of MMGAD with different weight factor β .

respectively. The best settings of α and β are used in the other experiments.

2) *Coefficient of Parameter Perturbation λ* : The coefficient of the parameter perturbation λ is used to control the amplitude of the parameter perturbation. A large λ indicates a large difference in the GNN functions, resulting in a large gap between the two graph embeddings generated for contrast. In this case, the multiview loss $\mathcal{L}_{mv}$ becomes large. In contrast, the distance between the embeddings for contrast is short and the loss is small. To find out the best settings of λ for these datasets, we compare the performance of MMGAD under different $\lambda \in \{0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5\}$, as shown in Fig. 8. When λ is set to 0, there is no difference between the perturbed module and the original module. In this case, the contrastive learning is completely ineffective. When λ is greater than 0.2, the performance shows a gradual downward trend as λ increases overall. We find that better performance is obtained when λ is in the range of 0.05–0.2. Theoretically, an appropriate λ forces MMGAD to capture latent patterns that are invariant to perturbation by contrastive learning. In contrast, excessive perturbations make it difficult to learn the globally optimal representations due to the strong oscillations in their gradients. Accordingly, we fixedly set $\lambda = 0.1$ for all datasets in our experiments.

3) *Normal Training Rate p and Selection Rate q* : The rate p for normal training is a hyperparameter that specifies the proportion of total training steps dedicated to normal training. This phase of training takes place prior to the cycle training, ensuring the model develops a foundational level of discrimination. A higher value of p means that fewer steps are allocated to cycle training. In contrast, a lower value of p implies that

Fig. 8. AUC values of MMGAD with different coefficient λ .Fig. 9. AUC values of MMGAD with different (a) normal training rate p and (b) selection rate q .

the model has a weaker discrimination ability before cycle training, which could influence the selection of normal nodes. If the process of selecting normal nodes is unreliable, cycle training will degrade into normal training. In this experiment, p is chosen from $p \in \{0, 0.25, 0.5, 0.75, 1\}$. As illustrated in Fig. 9(a), when p increases from 0 to 1, the AUC values generally show a slight upward trend at first, followed by a slight decline. Accordingly, we set $p = 0.5$ for all datasets in other experiments. The selection rate q is a hyperparameter for cycle training. We simply select q percent of the training nodes according to their anomaly scores. A larger q indicates that more samples are selected for an additional cycle training step. Since a small percentage of the anomalous nodes in the samples are mixed, we should set an appropriate q for each dataset to extract as many normal samples as possible. Considering the small percentage of anomalies and the basic discrimination capability of the model with half of the total training steps ($p = 0.5$), we set $q \in \{0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$. As shown in Fig. 9(b), there is a slight upward trend with the increase of q . In general, better performance can be achieved when q increases to 0.9 or 0.95. In practice, we set $q = 0.9$ for BlogCatalog and Flickr, and $q = 0.95$ for Cora, CiteSeer, UAI2010, and ACM.

4) *Subgraph Size and Hidden Dimension*: We further explore the sensitivity of the subgraph size m and the hidden dimension of the GNN modules. Fig. 10(a) shows the AUC values with different subgraph sizes. The subgraph size is selected from the set $m \in \{2, 4, 6, 8, 10\}$. As can be seen in the figure, $m = 4$ is a better choice for most of the datasets. If the size is set to $m = 2$, the performance drops on most datasets. In this case, there are only two nodes in a subgraph, which leads

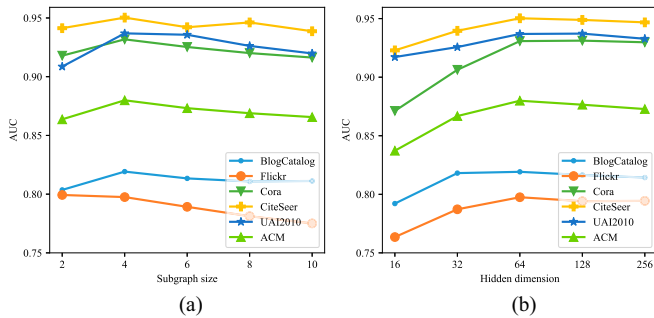


Fig. 10. AUC values of MMGAD with different (a) subgraph sizes and (b) hidden dimensions.

to a lack of structural information. However, the performance does not always improve with increasing size m . We argue that redundant information contained in a subgraph with excessive large size degrades the performance. The hidden dimension d_h is another parameter that affects the learning of graph representation. We compare the performance of MMGAD under different $d_h \in \{16, 32, 64, 128, 256\}$, as shown in Fig. 10(b). The performance improves with the increase of d_h when $d_h \leq 64$. However, when d_h is greater than 64, there is no improvement in performance. In fact, a slight decrease in AUC values is observed when d_h increases from 64 to 256. In addition, a larger d_h increases the computational cost and the risk of overfitting. In a comprehensive view, d_h is set to 64 for all datasets.

F. Discussion

MMGAD fully exploits the methods of multiview GCL and cycle training for node anomaly detection on attributed networks. Although the method has shown promising results, there are still some limitations and challenges that need to be addressed further.

1) *Limitations*: There are several limitations. First, the method introduces an additional graph-level contrastive learning branch and a cycle training stage, which leads to higher computational complexity and may affect its application for real-time or large-scale tasks. An area for future work will focus on improving computational efficiency. Second, the reliance on multiview contrastive learning and cycle training introduces additional hyperparameters that must be manually tuned, potentially creating challenges in contexts where anomalies are vaguely defined or extremely heterogeneous. Furthermore, the representation learning is closely tied to the structures and attributes, which could reduce its effectiveness in networks with inadequate or misleading attributes. One possible avenue for future research is to create a more streamlined and resilient framework against perturbations and noise.

2) *Opportunities*: Although the method is developed for node anomaly detection, it could potentially be adapted for edge anomaly detection through upgrades and modifications. One worthwhile approach is to replace the graph representation of MMGAD with a hypergraph representation, allowing edges in the network to be represented as nodes in the hypergraph. In this manner, the method could potentially be adapted to detect

anomalies on the edges. This method is not ideally suited for evolving networks because of the dynamic changes in both the network topology and the data flow. Integrating dynamic graph representation with incremental learning techniques could serve as a feasible solution.

VI. CONCLUSION

In this work, we propose MMGAD, a multiscale and multiview contrastive learning framework for anomaly detection on attributed networks. Unlike existing multiview methods that directly augment graph data to generate new graph views, our parameter perturbation-based network augmentation method generates new GNN views for graph-level contrast at multiscale. It is a simple yet efficient mechanism that improves the alignment and uniformity of graph embeddings without modifying the highly sensitive graph data. We also provide a cycle training strategy to alleviate the problem of learning normal patterns caused by the training data mixed with anomalous samples. The model is refined by additional training steps using selected normal samples in a circular manner. The experimental results demonstrate that MMGAD outperforms the baseline methods and that each designed module in MMGAD is effective. In the future, we plan to investigate the hypergraph learning method for GAD and extend the framework to support edge anomaly detection on attributed networks.

REFERENCES

- [1] W. Fan et al., "Graph neural networks for social recommendation," in *Proc. ACM Web Conf. (WWW)*, New York, NY, USA, 2019, pp. 417–426.
- [2] M. Gao, L. Wu, Q. Li, and W. Chen, "Anomaly traffic detection in IoT security using graph neural networks," *J. Inf. Secur. Appl.*, vol. 76, 2023, Art. no. 103532.
- [3] S. I. Popoola, B. Adebisi, M. Hammoudeh, G. Gui, and H. Gacamin, "Hybrid deep learning for botnet attack detection in the Internet-of-Things networks," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4944–4956, Mar. 2021.
- [4] A. Srivastava, A. Kundu, S. Sural, and A. Majumdar, "Credit card fraud detection using hidden Markov model," *IEEE Trans. Dependable Secure Comput.*, vol. 5, no. 1, pp. 37–48, Jan./Mar. 2008.
- [5] J. Yu et al., "Group-based fraud detection network on e-commerce platforms," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Min. (KDD)*, New York, NY, USA, 2023, pp. 5463–5475.
- [6] H. Zhang, K. Zeng, and S. Lin, "Federated graph neural network for fast anomaly detection in controller area networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1566–1579, 2023.
- [7] S. Rao, A. K. Verma, and T. Bhatia, "A review on social spam detection: Challenges, open issues, and future directions," *Expert Syst. Appl.*, vol. 186, 2021, Art. no. 115742.
- [8] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [9] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Inter. Conf. Data Mining (SDM)*, 2019, pp. 594–602.
- [10] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Conf. Neural Inf. Process. Syst. (NeurIPS)*, Red Hook, NY, USA: Curran Associates Inc., 2020, pp. 5812–5823.
- [11] M. Jin, Y. Liu, Y. Zheng, L. Chi, Y.-F. Li, and S. Pan, "Anemone: Graph anomaly detection with multi-scale contrastive learning," in *Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, New York, NY, USA, 2021, pp. 3122–3126.

- [12] C. Wang and H. Zhu, "Wrongdoing monitor: A graph-based behavioral anomaly detection in cyber security," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2703–2718, 2022.
- [13] J. Zhang, S. Wang, and S. Chen, "Reconstruction enhanced multi-view contrastive learning for anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 259–268.
- [14] F. Liu et al., "DAGAD: Data augmentation for graph anomaly detection," in *Proc. IEEE Int. Conf. Data Min. (ICDM)*, 2022, pp. 259–268.
- [15] J. Duan et al., "Normality learning-based graph anomaly detection via multi-scale contrastive learning," in *Proc. ACM Int. Conf. Multimedia*, 2023, pp. 7502–7511.
- [16] C. Niu, G. Pang, and L. Chen, "Graph-level anomaly detection via hierarchical memory networks," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases (ECML PKDD)*, D. Koutra, C. Plant, M. Gomez Rodriguez, E. Baralis, and F. Bonchi, Eds., 2023, pp. 201–218.
- [17] B. Perozzi and L. Akoglu, "Scalable anomaly ranking of attributed neighborhoods," in *Proc. SIAM Int. Conf. Data Min.*, 2016, pp. 207–215.
- [18] J. Li, H. Dani, X. Hu, and H. Liu, "Radar: Residual analysis for anomaly detection in attributed networks," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2017, pp. 2152–2158.
- [19] Z. Peng, M. Luo, J. Li, H. Liu, and Q. Zheng, "Anomalous: A joint modeling approach for anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3513–3519.
- [20] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.
- [21] M. Li et al., "Guest editorial: Deep neural networks for graphs: Theory, models, algorithms, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4367–4372, Apr. 2024.
- [22] C. Huang, M. Li, F. Cao, H. Fujita, Z. Li, and X. Wu, "Are graph convolutional networks with random weights feasible?," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2751–2768, Mar. 2023.
- [23] J. Duan et al., "Graph anomaly detection via multi-scale contrastive learning networks with augmented view," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, 2023, pp. 7459–7467.
- [24] S. Zhou, X. Huang, N. Liu, H. Zhou, F.-L. Chung, and L.-K. Huang, "Improving generalizability of graph anomaly detection models via data augmentation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12721–12735, Dec. 2023.
- [25] S. Qin, L. Yongcan, and T. Gaofeng, "Memory-augmented u-transformer for multivariate time series anomaly detection," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [26] Y. Lai, Y. Han, and Y. Wang, "Anomaly detection with prototype-guided discriminative latent embeddings," in *Proc. IEEE Int. Conf. Data Min. (ICDM)*, 2021, pp. 300–309.
- [27] Y. Liu et al., "Anomaly detection in dynamic graphs via transformer," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12081–12094, Dec. 2023.
- [28] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.
- [29] X. Luo et al., "ComGA: Community-aware attributed graph anomaly detection," in *Proc. ACM Int. Conf. Web Search Data Min. (WSDM)*, New York, NY, USA, 2022, pp. 657–665.
- [30] L. Huang et al., "Hybrid-order anomaly detection on attributed networks," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12249–12263, Dec. 2023.
- [31] L.-H. Chen et al., "AnomMAN: Detect anomalies on multi-view attributed networks," *Inf. Sci.*, vol. 628, pp. 1–21, 2023.
- [32] J. Liu, M. He, X. Shang, J. Shi, B. Cui, and H. Yin, "BOURNE: Bootstrapped self-supervised learning framework for unified graph anomaly detection," in *Proc. IEEE 40th Int. Conf. Data Eng. (ICDE)*, 2024, pp. 2820–2833.
- [33] J. Kim, Y. In, K. Yoon, J. Lee, and C. Park, "Class label-aware graph anomaly detection," in *Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2023, pp. 4008–4012.
- [34] Y. Zheng, M. Jin, Y. Liu, L. Chi, K. T. Phan, and Y.-P. P. Chen, "Generative and contrastive self-supervised learning for graph anomaly detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12220–12233, Dec. 2023.
- [35] K. Ding, J. Li, N. Agarwal, and H. Liu, "Inductive anomaly detection on attributed networks," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2020, pp. 1288–1294.
- [36] S. Zhou, Q. Tan, Z. Xu, X. Huang, and F.-I. Chung, "Subtractive aggregation for attributed network anomaly detection," in *Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 3672–3676.
- [37] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–14.
- [38] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–12.
- [39] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1025–1035.
- [40] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 6000–6010.
- [41] Z. Chen, D. Chen, X. Zhang, Z. Yuan, and X. Cheng, "Learning graph structures with transformer for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9179–9189, Jun. 2022.
- [42] M. Li, L. Zhang, L. Cui, L. Bai, Z. Li, and X. Wu, "BLoG: Bootstrapped graph representation learning with local and global regularization for recommendation," *Pattern Recogn.*, vol. 144, no. C, pp. 1–13, 2023.
- [43] J. Li, R. Zheng, H. Feng, M. Li, and X. Zhuang, "Permutation equivariant graph framelets for heterophilous graph learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 11634–11648, Sep. 2024.
- [44] L. Bai et al., "HAQJSK: Hierarchical-aligned quantum Jensen-Shannon kernels for graph classification," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6370–6384, Nov. 2024.
- [45] Z. Chai et al., "Can abnormality be detected by graph neural networks?," in *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 2022, pp. 1945–1951.
- [46] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [47] S. Qin, L. Chen, Y. Luo, and G. Tao, "Multiview graph contrastive learning for multivariate time-series anomaly detection in IoT," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22401–22414, Dec. 2023.
- [48] K. Ding, Z. Xu, H. Tong, and H. Liu, "Data augmentation for deep graph learning: A survey," *SIGKDD Explor. Newsl.*, vol. 24, no. 2, pp. 61–77, Dec. 2022.
- [49] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Graph contrastive learning with adaptive augmentation," in *Proc. ACM Web Conf. (WWW)*, 2021, pp. 2069–2080.
- [50] Y. Liu et al., "Simple contrastive graph clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13789–13800, Oct. 2024.
- [51] J. Xia, L. Wu, J. Chen, B. Hu, and S. Z. Li, "SimGRACE: A simple framework for graph contrastive learning without data augmentation," in *Proc. ACM Web Conf. (WWW)*, 2022, pp. 1070–1079.
- [52] H. Tong, C. Faloutsos, and J.-y. Pan, "Fast random walk with restart and its applications," in *Proc. IEEE Int. Conf. Data Min. (ICDM)*, 2006, pp. 613–622.
- [53] M. Sun, J. Xing, H. Wang, B. Chen, and J. Zhou, "MoCL: Data-driven molecular fingerprint via knowledge-aware contrastive learning from molecular graph," in *Proc. ACM SIGKDD Conf. Knowl. Disc. Data Min. (KDD)*, 2021, pp. 3585–3594.
- [54] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 9929–9939.
- [55] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Nov. 2021, pp. 6894–6910.
- [56] L. Tang and H. Liu, "Relational learning via latent social dimensions," in *Proc. ACM SIGKDD Conf. Knowl. Disc. Data Min. (KDD)*, 2009, pp. 817–826.
- [57] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, "Collective classification in network data articles," *AI Mag.*, vol. 29, pp. 93–106, Sep. 2008.
- [58] W. Wang, X. Liu, P. Jiao, X. Chen, and D. Jin, "A unified weakly supervised framework for community detection and semantic matching," in *Proc. Pacific-Asia Conf. Knowl. Disc. Data Min.*, 2018, pp. 218–230.
- [59] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "Arnet-Miner: Extraction and mining of academic social networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Min. (KDD)*, 2008, pp. 990–998.
- [60] A. Paszke et al., "Automatic differentiation in PyTorch," in *Proc. NIPS-W*, 2017, pp. 1–4.